

C++内存模型与名称空间练习

C++ 内存模型与名称空间 练习题及答案

一、选择题（基础概念）

1. 以下哪项是头文件中不允许包含的内容？

- A. 结构体声明
- B. 函数原型
- C. 非内联函数定义
- D. #define 常量

答案：C

解析：头文件应仅包含声明（如原型、结构体），非内联函数定义会导致多文件编译时的重复定义错误。

2. 关于 `static` 全局变量，以下说法正确的是？

- A. 具有外部链接性，可被其他文件访问
- B. 具有内部链接性，仅当前文件可见
- C. 存储在栈中，随函数结束释放
- D. 未初始化时默认值为随机值

答案：B

解析：`static` 全局变量的链接性为内部，作用域限于当前文件，未初始化时默认零值。

3. 动态内存分配失败时，C++ 现代实现会？

- A. 返回 `nullptr`
- B. 抛出 `std::bad_alloc` 异常
- C. 终止程序
- D. 分配部分内存

答案：B

解析：C++11 后 new 失败默认抛异常，需用 try-catch 或 new (nothrow) 处理。

4. using namespace std; 与 using std::cout; 的主要区别是？

- A. 前者引入单个成员，后者引入整个命名空间
- B. 后者会导致命名冲突，前者不会
- C. 前者可能静默覆盖局部变量，后者强制报错
- D. 无区别，只是写法不同

答案：C

解析：using 指令 引入整个命名空间，局部变量同名时静默覆盖；using 声明 引入单个成员，冲突时编译器报错。

二、简答题（核心原理）

1. 简述 extern int x; 和 int x; 的区别。

答案：

- extern int x; 是**声明**，告诉编译器 x 在别处定义，不分配内存。
- int x; 是**定义**，分配内存并初始化（默认零值），只能在一个文件中出现。

2. 为什么动态数组释放必须用 delete[]?

答案：

- delete[] 会调用每个元素的析构函数（针对类对象），并正确释放整个数组内存。
- 误用 delete 会导致仅释放首元素内存，其余元素内存泄漏（基本类型无析构函数，但仍需 delete[] 保证行为正确）。

3. 简述智能指针 unique_ptr 和 shared_ptr 的核心区别。

答案：

- unique_ptr：独占所有权，不可拷贝，离开作用域自动释放（适合独占资源）。
- shared_ptr：共享所有权，引用计数归零时释放，支持拷贝（适合多所有者场景）。
- 核心区别：所有权管理方式（独占 vs 共享）。

三、编程题（实践应用）

1. 多文件程序设计

设计一个 `golf` 结构体，包含姓名（`string`）和差点（`int`），实现以下功能：

答案框架：

```
// golf.h
#ifndef GOLF_H
#define GOLF_H
#include <string>
struct Golf {
    std::string fullname;
    int handicap;
};
bool setGolf(Golf& g);    // 交互式输入
void setGolf(Golf& g, const std::string& name, int hc); // 非交互式
void showGolf(const Golf& g);
#endif
// golf.cpp
#include <iostream>
#include "golf.h"
bool setGolf(Golf& g) {
    std::cout << "Enter name: ";
    std::getline(std::cin, g.fullname);
    if (g.fullname.empty()) return false;
    std::cout << "Enter handicap: ";
    std::cin >> g.handicap;
    std::cin.ignore(); // 清除换行符
    return true;
}
void setGolf(Golf& g, const std::string& name, int hc) {
    g.fullname = name;
    g.handicap = hc;
}
void showGolf(const Golf& g) {
    std::cout << "Name: " << g.fullname
               << ", Handicap: " << g.handicap << "\n";
}
```

```
// main.cpp
#include <iostream>
#include "golf.h"
const int SIZE = 10;
int main() {
    Golf golfers[SIZE];
    int count = 0;
    while (count < SIZE && setGolf(golfers[count])) {
        count++;
    }
    for (int i = 0; i < count; i++) {
        showGolf(golfers[i]);
    }
    return 0;
}
```

- 头文件 `golf.h` 声明结构体和函数原型（`setGolf` 初始化、`showGolf` 显示）。
- 源文件 `golf.cpp` 实现函数，`setGolf` 支持手动输入或参数初始化。
- `main.cpp` 中创建数组，循环输入直到姓名为空。

2. 动态内存与定位 `new`

使用定位 `new` 在静态缓冲区中创建两个 `chaff` 对象（`chaff` 包含 `char[20]` `dross` 和 `int` `slag`），输入数据并显示。

答案框架：

```
#include <iostream>
#include <new>
struct chaff {
    char dross[20];
    int slag;
};
const int BUF = 512;
char buffer[BUF];
int main() {
    chaff* p1 = new (buffer) chaff;
    chaff* p2 = new (buffer + sizeof(chaff)) chaff;
    std::cout << "Enter dross for p1: ";
    std::cin.getline(p1->dross, 20);
    std::cout << "Enter slag for p1: ";
    std::cin >> p1->slag;
    std::cin.ignore();
    std::cout << "Enter dross for p2: ";
```

```
std::cin.getline(p2->dross, 20);
std::cout << "Enter slag for p2: ";
std::cin >> p2->slag;
std::cout << "\nChaff 1: " << p1->dross << ", " << p1->slag << "\n";
std::cout << "Chaff 2: " << p2->dross << ", " << p2->slag << "\n";
return 0;
}
```

四、陷阱题（易错点）

1. 以下代码是否正确？说明原因。

```
// header.h
#ifndef HEADER_H
#define HEADER_H
int shared = 100; // 全局变量定义
#endif
// file1.cpp
#include "header.h"
// file2.cpp
#include "header.h"
```

答案：错误。

解析：头文件中定义全局变量 `shared`，导致 `file1.cpp` 和 `file2.cpp` 包含后出现重复定义，链接器报错。应将 `shared` 声明为 `extern` 并在一个源文件中定义。

2. 分析内存泄漏风险：

```
void func() {
    int* p = new int[10];
    // ... 使用 p ...
    delete p; // 释放单个指针
}
```

答案：内存泄漏。

解析：动态数组需用 `delete[]` 释放，误用 `delete` 仅释放首元素，其余 9 个元素内存泄漏。

五、设计题（综合应用）

设计一个 `Sales` 类，记录季度销售额，要求：

- 使用动态内存存储销售额（长度可变）。
- 包含计算平均、最大、最小值的方法。
- 使用智能指针管理内存，避免泄漏。

答案要点：

```
#include <memory>
#include <vector>
class Sales {
private:
    std::unique_ptr<double[]> sales;
    int quarters;
public:
    Sales(int q) : quarters(q), sales(new double[q]{0}) {}
    void setSale(int index, double val) { sales[index] = val; }
    double average() const {
        double sum = 0;
        for (int i = 0; i < quarters; i++) sum += sales[i];
        return sum / quarters;
    }
    // 省略 max、min 实现...
};
```

总结：关键考点

1. **头文件规范**：声明 vs 定义，`#ifndef` 防护。
2. **存储类型**：`static` 链接性，`extern` 的作用。
3. **动态内存**：`new/delete` 配对，数组释放，定位 `new` 的使用。
4. **名称空间**：`using` 声明 vs `using` 指令，冲突处理。
5. **智能指针**：`unique_ptr` 与 `shared_ptr` 的适用场景。

通过练习掌握内存管理的安全性和命名空间的最佳实践，避免常见错误（如内存泄漏、命名冲突）。

（注：文档部分内容可能由 AI 生成）